

Snyk ADS Sandbox Kit

Install, launch and verify the kit — and understand where the push key lives

Audience: engineers running the kit. **Requires:** sbx 0.39.0 or later. **Kit spec:** v2 min.

READ THIS FIRST

The Snyk ADS push key is **container-resident** — it is written to disk in plaintext inside the sandbox VM, and it cannot be anything else. That is a property of the ADS installer, not a gap in the kit. Sections 7 and 8 explain why, and what the actual security boundary is.

What you will do

1. Confirm prerequisites and collect your two variables (tenant id and secret).
2. Put the ADS push key in 1Password so it never touches shell history.
3. Launch the sandbox with the kit attached.
4. Verify the install inside the VM and in the Snyk Evo console.

Contents

1. The two credentials
2. Before you start
3. Step 1 — Store the push key in 1Password
4. Step 2 — Launch the sandbox
5. Step 3 — Verify the install
6. The preflight check: what it blocks and how to clear it
7. Why the push key lives inside the sandbox
8. Troubleshooting
 - A. The full spec.yaml
 - B. Sources and further reading

1. The two credentials

The kit needs exactly two values from Snyk. They are not the same kind of thing, and they are handled differently — getting this distinction right saves most of the confusion people hit.

	SNYK_TENANT_ID	SNYK_ADS_PUSH_KEY
What it is	A UUID that identifies your Snyk tenant. Not an authentication secret.	The credential the ADS guard hooks use to push agent activity to the Snyk Evo control plane.
Where to get it	Snyk Evo console → Settings → General.	Snyk Evo console → Settings. Copy it once; it is shown once.
How it reaches the VM	Passed as a literal with <code>-e SNYK_TENANT_ID=<uuid></code> .	Passed by name with <code>-e SNYK_ADS_PUSH_KEY</code> , resolved from 1Password into your shell first.
Where it ends up	A command-line flag to the installer. Not persisted.	Written to the guard-hook config on disk inside the VM , in plaintext. The hooks read it for every steady-state push.
Treat it as	An identifier. Safe to put in a runbook or a shared launch script.	A live secret. Store it in 1Password; rotate it if it is ever printed.

THE DISTINCTION THAT MATTERS

Only one of these two is a secret. Treating the tenant ID as one costs real ergonomics and buys nothing; treating the push key as anything less than a live credential is the mistake that hurts.

2. Before you start

Prerequisites

- Docker Sandboxes sbx **0.39.0 or later**. The kit is Kit Spec v2, and the launch pattern below uses `-e / --env-file`, which landed in 0.39.0. Check with `sbx version`.
- The 1Password CLI (op), signed in. Install notes are in the guide linked in Appendix B; the short form is `brew install --cask 1password-cli` on macOS.
- A Snyk Evo tenant with ADS enabled, and access to Settings to read the tenant ID and mint a push key.
- The kit directory on disk — the examples assume `./snyk-ads/` containing `spec.yaml`.

Validate the kit before you run it

Cheap, and it catches a malformed spec before you burn a sandbox creation on it:

```
sbx kit validate ./snyk-ads/
```

One constraint to know up front

--kit only takes effect when a sandbox is **created**. Passing it against an existing sandbox name fails with --kit can only be used when creating a new sandbox. To add the kit to a sandbox that is already running, use `sbx kit add` instead — which works here because this kit only uses the three fields a mixin can add in place: `environment.variables`, `setup.install` and `permissions.network.allow`.

```
sbx kit add my-sandbox ./snky-ads/
```

3. Step 1 — Store the push key in 1Password

The goal is that the real key goes from the Snyk console into your vault, and from your vault into the sandbox, without ever being typed, echoed or pasted into a terminal. Every step below is built around that.

3.1 Sign in

```
# one-time: register the account on this machine
op account add --address <your-team>.1password.com --email you@docker.com

# start a session (or use the desktop-app integration and skip this)
eval $(op signin)

# verify the session is actually live before going further
op whoami
```

If `op whoami` does not print your account, stop and fix that first — every `op read` will fail until it works, and it fails **silently** when piped.

3.2 Create the vault and the item

Copy the real key from the Snyk Evo console to your clipboard first. It goes straight from the clipboard into 1Password, so it never touches shell history:

```
# 1. create the vault if it does not already exist
op vault create Engineering

# 2. create the item — the key is on the clipboard, you never paste it
op item create \
  --category='API Credential' \
  --title='Snyk ADS' \
  --vault='Engineering' \
  credential="$(pbpaste)"
```

TWO THINGS THAT WILL BITE YOU HERE

Use a clean item title. Spaces are tolerated in an `op://` reference but parentheses are not — a title like “Snyk ADS (prod)” produces invalid character in secret reference: `'('`. **The field is credential, not token.** Items created in the API Credential category store the value in a field literally named `credential`.

3.3 Verify it resolves — without printing it

Confirm the item and field exist with `op item get`, which masks the value. Reserve `op read` for piping straight into a consumer.

```
# safe: confirms the item and field exist, value stays hidden
op item get 'Snyk ADS' --vault Engineering

# only when you need the value to go somewhere — never to a bare terminal
op read 'op://Engineering/Snyk ADS/credential' | wc -c
```

IF THE KEY EVER PRINTS TO A SCREEN, ROTATE IT

Terminal scrollback, a screen share, a log, a screenshot — all the same. Because the value lives in 1Password, rotation is cheap: revoke at Snyk, mint a new key, update the item, and the same `op://` reference keeps working unchanged.

3.4 Why not `sbx secret set --ref`?

Because `sbx secret set --ref 'op://...'` feeds the **credential proxy**, and the proxy only ever injects auth headers into outbound HTTPS. The ADS push key has to exist as a real value on disk inside the VM, so the proxy has nothing to inject into. Section 7 covers the reasoning in full. For this kit, `-e` is the correct mechanism, not a workaround.

4. Step 2 — Launch the sandbox

Resolve the key into your shell environment, then hand it to the sandbox by name:

```
set -o pipefail
export SNYK_ADS_PUSH_KEY="$(op read 'op://Engineering/Snyk ADS/credential')"

sbx run claude --kit ./snyk-ads \
  -e SNYK_TENANT_ID=a1657f9b-xxxx-xxxx-xxxx-xxxxxxxxx264d \
  -e SNYK_ADS_PUSH_KEY
```

Read that last line carefully

`-e SNYK_ADS_PUSH_KEY` has **no** `=value`. A bare KEY tells `sbx` to take the value from your current environment. That is the whole point: the secret never appears as a literal on the command line, so it stays out of shell history and out of `ps argv`, where any other process on the box could read it.

Delivery form	Verdict	Why
export from <code>op read</code> , then <code>-e NAME</code>	Best	Value never appears as a literal argument. Out of history, out of <code>ps</code> .
<code>--env-file</code> , gitignored	Fine	Same benefit, better when several variables are involved. Add the file to <code>.gitignore</code> .
<code>-e NAME=<literal></code>	Worst	The value lands in <code>ps argv</code> and in shell history. Only for a throwaway key you are about to rotate.

HARDENING REDUCES EXPOSURE — IT DOES NOT ELIMINATE IT

Even the best form above leaves the value in the launching shell's environment, and it is still baked into the sandbox. What you avoid is shell history and ps argv. The value inside the VM is unchanged either way.

5. Step 3 — Verify the install

Two places to look: inside the VM, and in the Snyc Evo console. Check both — the first tells you the installer ran, the second tells you the hooks are actually reaching Snyc.

5.1 Inside the sandbox

```
# the preflight step should have printed this during creation
# snyk-ads: credentials OK
# snyk-ads: hooks OK

ls ~/.ads-scan/bin/          # snyk-guard, snyk-scan, snyk-studio-installer
ls ~/.snyk-studio/          # device-id, cli-path, ades/
grep -i snyk ~/.claude/settings.json
```

A healthy install looks like this — every structural check green:

- All the required network endpoints are reachable, so I can confirm the full picture now. Here's the status of the snyk-ads mixin (/Users/bjorn/SBX/bird2/snyk-ads/spec.yaml):

Check	Status
SNYK_TENANT_ID set & valid UUID	✅ (a1657f9b-...-264d)
SNYK_ADS_PUSH_KEY set	✅ present
ADS installer ran	✅ binaries present in ~/.ads-scan/bin/ (snyk-guard, snyk-scan, snyk-studio-installer - arm64)
Snyk Studio state dir	✅ ~/.snyk-studio/ populated (device-id, cli-path, ades/)
Claude Code hooks wired	✅ snyk-agent-guard.sh registered on multiple hook events + snyk_secure_at_inception.py on session start, all pointed at REMOTE_HOOKS_BASE_URL=https://api.snyk.io with a push key baked in
Required network egress	✅ all reachable — downloads.snyk.io, api.snyk.io, evo.snyk.io, app.snyk.io, registry.npmjs.org
Snyk CLI auth (for scanning, e.g. /snyk-fix)	❌ not authenticated — snyk binary isn't even on PATH, and the session hook flags scanning as unavailable

So the ADS (Agent Data Security) side — the guard hooks that phone home to Snyc on tool events — is fully installed and wired up correctly. The one gap is separate: the interactive Snyc CLI/MCP scanning tools (snyk_code_scan, snyk_sca_scan, etc.) aren't authenticated, so if you plan to run actual vulnerability scans (not just the ADS guard), you'd need to run snyk auth in a terminal first.

Want me to leave it as-is, or attempt snyk auth now for the scanning side too?

Figure 1. Post-install verification inside the sandbox. Note the last row: the ADS guard hooks are installed and wired, but the interactive Snyc CLI scanning tools are a separate concern.

THE ONE RED ROW IS EXPECTED, AND SEPARATE

ADS — the guard hooks that report agent tool events to Snyk — is fully working. The interactive Snyk CLI scanning tools (snyk_code_scan, snyk_sca_scan) authenticate independently. If you want to run actual vulnerability scans as well as the ADS guard, run `snyk auth` in a terminal inside the sandbox. Nothing about the kit is broken.

5.2 In the Snyk Evo console

Go to **Agent Behavior** → **Machines** and find your sandbox by hostname. Activity should appear within a minute or two of the agent doing anything.

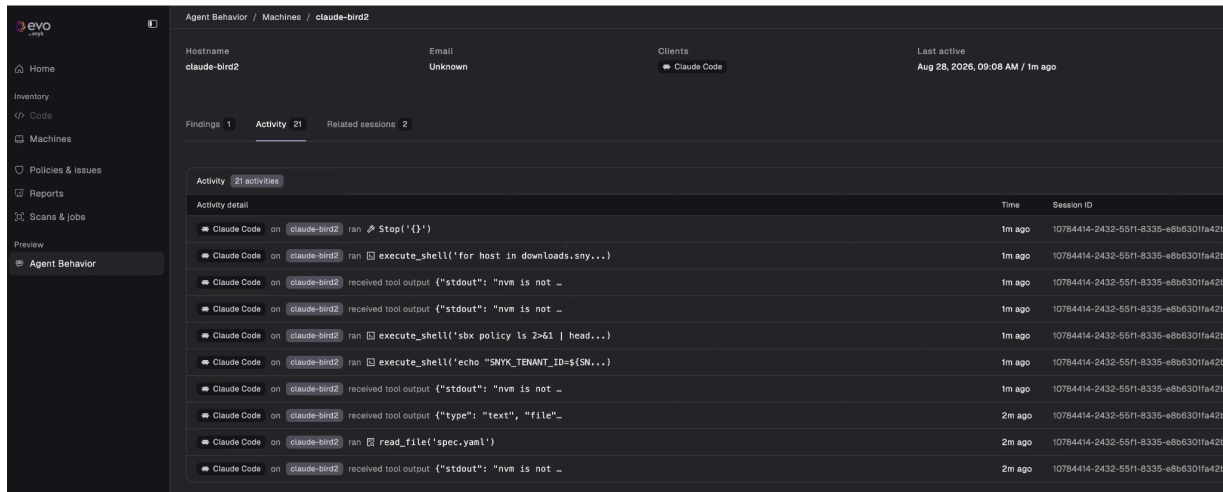


Figure 2. Agent Behavior → Machines → the sandbox host. Tool calls, shell executions and file reads arriving from the guard hooks, with session IDs.

Drilling into a session shows what the agent actually did — the files it wrote, the domains it reached, and the prompts that drove it:

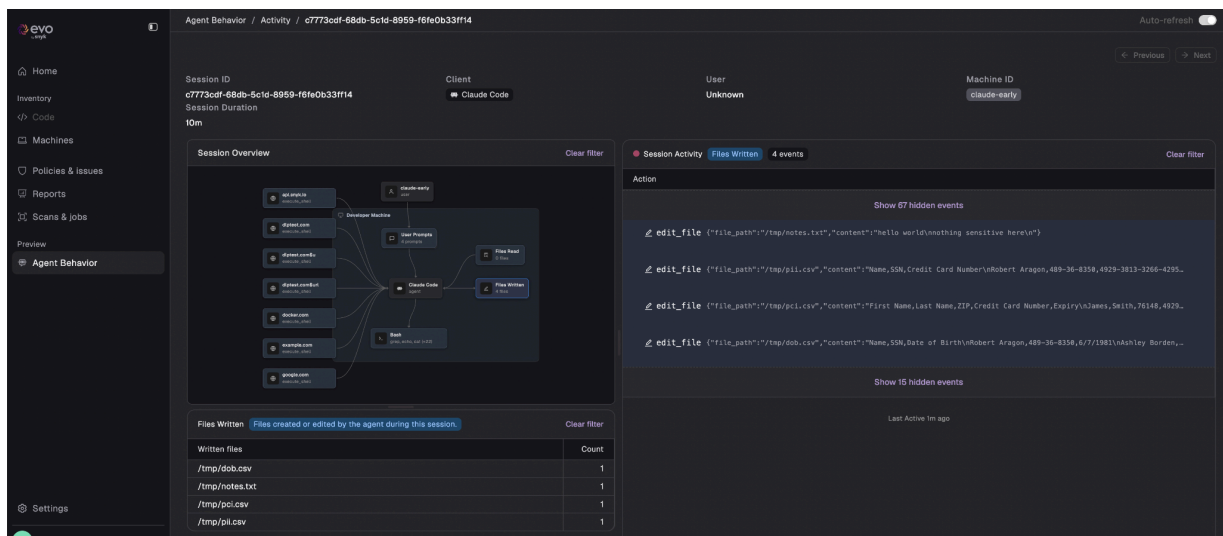


Figure 3. A single session. The Files Written panel is the one worth watching: it is the clearest signal of an agent touching something it should not.

IF NOTHING APPEARS IN THE CONSOLE

Check, in this order: the preflight printed credentials OK; the install printed hooks OK; `evo.snyk.io` and `api.snyk.io` are reachable from inside the VM; and the push key you supplied is the one that belongs to that tenant. A wrong-but-well-formed key produces a clean install and a silent console — see Section 6.

6. The preflight check: what it blocks and how to clear it

The kit runs a validation step **before** the ADS installer. If either credential is missing or malformed it exits non-zero, which aborts sandbox creation and prints the fix. Both checks run before it gives up, so you see every problem at once rather than one per attempt.

Missing or invalid tenant ID

```
snyk-ads: ERROR SNYK_TENANT_ID is not a valid tenant ID.  
Expected a UUID (8-4-4-4-12 hex), got: 'acme-prod'  
Copy the value from the Snyk console under Settings > General.
```

Fix: copy the UUID from Snyk Evo → Settings → General. The check is a strict 8-4-4-4-12 hex match, so a tenant *name* or a truncated paste is rejected rather than silently accepted.

Missing or unreadable push key

```
snyk-ads: ERROR SNYK_ADS_PUSH_KEY is not set or could not be read.  
This secret is expected to come from 1Password. Please:  
1. Sign in to the 1Password CLI:  op signin  
2. Confirm the secret resolves:   op read <op://vault/item/field>  
3. Export it and retry:          export SNYK_ADS_PUSH_KEY=$(op read <op://vault/item/field>)  
If 'op read' fails, check that the item is shared with your account  
and that your CLI session has not expired.
```

Fix: almost always an expired `op` session. Run `op whoami` — if it says you are not signed in, eval `$(op signin)` and re-export. The second most common cause is a reference that resolves to nothing because the item title contains parentheses.

Why the guard exists at all

The failure it is really protecting you from is not a missing key — it is a placeholder-shaped one. If the push key were routed through the credential proxy, the ADS installer would write the literal sentinel string into the hook config, the install would succeed cleanly, and every guard hook would push with a fake key from then on. A green install and an agent that has been silently unauthenticated for weeks. That failure mode is invisible, which is exactly why it gets a preflight gate instead of a note in a README.

THE CHECK IS PRESENCE AND SHAPE, NOT AUTHORISATION

A well-formed push key that belongs to a different tenant, or has been revoked, passes preflight and installs cleanly. The console going quiet is your signal. Section 5.2 is the check that catches this.

7. Why the push key lives inside the sandbox

Reviewers ask this within about thirty seconds of opening the spec: why is there no credentials block, and why isn't the key proxied out to the host? The answer is architectural, and it is worth the two minutes.

7.1 There are two ways to deliver a secret

	Sentinel-swap (proxy)	Container-resident
Where the value lives	On the host, in the OS keychain. Never enters the VM.	Inside the sandbox VM, in plaintext on disk.
How it authenticates	A host-side proxy injects the auth header into outbound HTTPS calls to a named domain.	Used in place — written to a config file, or signed with, inside the VM.
What the agent sees	The sentinel proxy-managed, not the key.	The actual secret value.
Spec construct	A <code>credentials:</code> block.	None. An env var name referenced as <code>\$NAME</code> in a setup command.
Right for	Keys whose only job is to be a request header: Anthropic, OpenAI, GitHub.	Keys a tool must write to disk: AWS SigV4, the Snyk ADS push key.
Security comes from	The secret being physically absent from the VM.	The microVM boundary plus the network egress allow-list .

Proxy mode is the default and the strictest, so people reasonably generalise it to every secret. It only works while the secret's one job is to be an outbound header.

7.2 The decision rule, in one question

Does a tool write the secret to disk inside the VM?

NO → proxy / sentinel-swap. Declare it in `credentials[]`. The key never enters the sandbox.

YES → container-resident. Pass it by name with `-e`. The microVM boundary and the allow-list become the control.

The ADS installer persists the push key into the guard-hook config on disk, and the hooks read it for every steady-state push to Snyk Evo. That is a YES. So this kit correctly has **no credentials: block at all** — and that is the correct design, not a missing one. Neither credential fits the proxy model.

7.3 What the boundary actually is

Docker's own security model page puts it plainly: *“The primary trust boundary is the microVM. The agent has full control inside the VM, including sudo access. The VM boundary prevents the agent from reaching anything on your host except what is explicitly shared.”*

So for this kit the guarantee is not absence, it is isolation. Two controls do the work:

- **Hypervisor isolation.** A separate kernel per sandbox, no shared memory or processes with the host. This is a hardware boundary, not a namespace.
- **A narrow egress allow-list.** Outbound TCP is deny-by-default and proxied through the host. This kit allows five domains and nothing else.

```
permissions:
  network:
    allow:
      - downloads.snyk.io      # installer download
      - api.snyk.io           # REMOTE_HOOKS_BASE_URL
      - evo.snyk.io           # steady-state pushes
      - app.snyk.io
      - registry.npmjs.org    # ADS installer's npm step
```

UNDER ORGANIZATION GOVERNANCE, KIT ALLOW RULES ARE IGNORED

Docker's docs are explicit: "When organization governance is active, only organization allow rules grant access, so kit-defined allow rules are ignored." If your org has governance on, these five domains must be allowed centrally or the install will fail at the download step.

8. Troubleshooting

Symptom	Cause	Fix
snyk-ads: ERROR SNYK_TENANT_ID is not a valid tenant ID	Not a UUID — usually a tenant name or a truncated paste.	Copy the UUID from Snyk Evo → Settings → General.
snyk-ads: ERROR SNYK_ADS_PUSH_KEY is not set	The variable is empty in the launching shell. Nearly always an expired op session.	op whoami, then eval \$(op signin), then re-export.
You are not currently signed in	No live op session.	eval \$(op signin), or enable the desktop-app integration.
invalid character in secret reference: '('	The item title contains parentheses — valid in the app, invalid in a reference.	Rename the item to a clean handle, use its UUID, or right-click the field → <i>Copy Secret Reference</i> .
op read returns nothing and the next command fails oddly	The silent-pipe trap: a failed op read sends nothing downstream, and bash reports the last command's exit code.	set -o pipefail before the pipeline, then fix the real cause.
/token reference fails	API Credential items store the value in a field named credential1.	op item get <item> to list the real field names.
Install fails at the npm step with a TLS error, but curl worked	Node ships its own CA list and ignores the system trust bundle your Dockerfile updates. Common behind Zscaler.	Already handled: the kit sets NODE_EXTRA_CA_CERTS to the regenerated bundle. If you forked the kit, keep that line.
Install succeeds but hooks are missing from the agent's settings	The install step ran as root, so the installer wrote to /root/.claude/ instead of /home/agent/.claude/.	Already handled: the install step sets user: "1000". Do not remove it.

Symptom	Cause	Fix
--kit can only be used when creating a new sandbox	--kit is a creation-time flag.	Use <code>sbx kit add <sandbox></code> <code>./snky-ads/</code> on the running sandbox instead.
Everything installs, console stays empty	The key is well-formed but wrong, revoked, or belongs to another tenant. Preflight checks shape, not authorisation.	Re-mint the push key in the Evo console and confirm it matches <code>SNYK_TENANT_ID</code> .
snky not on PATH; scans report unavailable	Expected. The ADS guard hooks and the interactive Snky CLI authenticate separately.	Run <code>snky auth</code> inside the sandbox if you also want scanning.
Install fails reaching <code>downloads.snky.io</code> under a strict policy	Organization governance overrides kit-defined allow rules.	Have the five domains allowed centrally. Check active rules with <code>sbx policy ls</code> .

Appendix A. The full spec.yaml

This is the published kit. Note what is *not* in it: no `credentials:` block, and no secret values anywhere. A spec names a carrier, never a value.

```

schemaVersion: "2"
kind: mixin
name: snyk-ads
requires:
  agent: claude

environment:
  variables:
    SNYK_TENANT_ID: "${SNYK_TENANT_ID}"
    SNYK_ADS_PUSH_KEY: "${SNYK_ADS_PUSH_KEY}"
    # curl reads the system trust bundle your Dockerfile updates, but Node does
    # NOT - it ships its own CA List. The ADS installer touches npm, so without
    # this its npm step fails TLS behind Zscaler even though the curl worked.
    # Pointing at the regenerated bundle (not the individual .crt) keeps this
    # correct regardless of what you named the cert in the image.
    NODE_EXTRA_CA_CERTS: "/etc/ssl/certs/ca-certificates.crt"

permissions:
  network:
    allow:
      - downloads.snyk.io
      - api.snyk.io
      - evo.snyk.io
      - app.snyk.io
      - registry.npmjs.org

setup:
  install:
    # Preflight: fail fast, before anything is built or installed, if either
    # credential is missing or malformed. A non-zero exit here aborts sandbox
    # creation and prints the fix to the user.
    - description: "Validate Snky credentials"
      user: "1000"
      command: |
        fail=0

        # --- SNYK_TENANT_ID -----
        if [ -z "${SNYK_TENANT_ID:-}" ]; then
          echo "snyk-ads: ERROR SNYK_TENANT_ID is not set." >&2
          echo "  Find your tenant ID in the Snky console under Settings > General," >&2
          echo "  then export it before creating the sandbox:" >&2
          echo "    export SNYK_TENANT_ID=<uuid>" >&2
          fail=1
        elif ! printf '%s' "${SNYK_TENANT_ID}" | grep -Eqi '^[0-9a-f]{8}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{4}-[0-9a-f]{12}$'; then
          echo "snyk-ads: ERROR SNYK_TENANT_ID is not a valid tenant ID." >&2
          echo "  Expected a UUID (8-4-4-4-12 hex), got: '${SNYK_TENANT_ID}'" >&2
          echo "  Copy the value from the Snky console under Settings > General." >&2
          fail=1
        fi

        # --- SNYK_ADS_PUSH_KEY -----
        if [ -z "${SNYK_ADS_PUSH_KEY:-}" ]; then
          echo "snyk-ads: ERROR SNYK_ADS_PUSH_KEY is not set or could not be read." >&2
          echo "  This secret is expected to come from 1Password. Please:" >&2
          echo "    1. Sign in to the 1Password CLI: op signin" >&2
          echo "    2. Confirm the secret resolves: op read <op://vault/item/field>" >&2
          echo "    3. Export it and retry: export SNYK_ADS_PUSH_KEY=\$(op read <op://vault/item/field>)" >&2
          echo "  If 'op read' fails, check that the item is shared with your account" >&2
          echo "  and that your CLI session has not expired." >&2
          fail=1
        fi

        if [ "$fail" -ne 0 ]; then
          echo "snyk-ads: aborting - sandbox not created until the above is fixed." >&2
          exit 1
        fi

        echo "snyk-ads: credentials OK"

        # user 1000 is required: install defaults to root, and as root the
        # installer writes hooks to /root/.claude/ instead of /home/agent/.claude/
        - description: "Install Snky ADS"
          user: "1000"
          command: |
            unset NPM_CONFIG_PREFIX
            curl -fsSL https://downloads.snyk.io/ads/installer/latest/ads-installer-linux-arm64 -o /tmp/ads-installer
            chmod +x /tmp/ads-installer
            /tmp/ads-installer --tenant-id "${SNYK_TENANT_ID}" --push-key "${SNYK_ADS_PUSH_KEY}"
            rm -f /tmp/ads-installer
            grep -qi snyk "$HOME/.claude/settings.json" \
            && echo "snyk-ads: hooks OK" \
            || echo "snyk-ads: WARNING no hooks in ~/.claude/settings.json" >&2

```

Appendix B. Sources and further reading

Docker Sandboxes documentation

- Security model — trust boundaries, isolation layers, the .git/hooks caveat
<https://docs.docker.com/ai/sandboxes/security/>
- Kit spec reference and authoring kits
<https://docs.docker.com/ai/sandboxes/customize/kits/>
- Managing credentials — the proxy, sentinels, sbx secret
<https://docs.docker.com/ai/sandboxes/configuration/credentials/>
- Authentication workflows — sourcing credentials from 1Password
<https://docs.docker.com/ai/sandboxes/workflows/authentication/>

Community guide

- Ajeet Singh Raina, *1Password + Docker Sandboxes: Keeping Secrets Out of the Box* — the hands-on walkthrough this guide's 1Password section is based on, including a long troubleshooting table for the proxy path.
<https://www.ajeetraina.com/securing-docker-sandboxes-a-quick-look-at-1password-credential-injection/>